University of Science and Technology of Oran Mohamed Boudiaf
Faculty of Mathematics and Informatics
Department of Computer Science

Collaboration and quality

Pull requests, code review and tests

Mohammed Said Belaid
Eng 5 SECI, DevOps

Opening

Today's agenda

1. Production is broken, again
2. Why other eyes? Code review
3. GitHub flow and pull requests
4. Code review in practice
5. Why tests?
6. Writing tests with pytest
7. Wrap-up

> Today's question: *why* should a change be read by someone else and checked by a machine before it reaches `main`?

Production is broken, again

```
def apply_discount(prices, percent):
    return [p * (1 - percent / 100)
            for p in prices]
```

- A promotion is configured at 150 instead of 15
- `apply_discount([100], 150)` → `[-50.0]`

Production is broken, again

```
def apply_discount(prices, percent):
    return [p * (1 - percent / 100)
            for p in prices]
```

- A promotion is configured at 150 instead of 15
- `apply_discount([100], 150)` → `[-50.0]`

- Prices become negative
- For one hour, the shop pays its customers to take its products
- The commit was pushed directly to `main`, the evening before
- "I tried it with 10%, it worked."

> The code was in Git, with a good commit message. History tells us *what* happened. It did not stop it.

Two questions

1. Who else looked at this change?

2. How do we know it works?

Two questions

1. Who else looked at this change?

2. How do we know it works?

Question	Practice	Tool today
Who else looked at it?	Code review	Pull requests
How do we know it works?	Automated tests	pytest
Can someone skip both?	Protected `main`	Branch rules

Why other eyes? Code review

Why review code?

What a reviewer catches

- Bugs and missing cases ("what if the cart is empty?")
- Unclear names, duplicated code
- Design problems, security holes
- Missing tests

What the team gains

- Two people know each part of the code
- Juniors learn from seniors, and the reverse
- Shared style and conventions
- A record of why, in the discussion

> Review is not a judgement of the author: it is the team taking shared ownership of the code (lecture 1: blameless culture).

What the evidence says

Technique (McConnell, *Code Complete*, 2004)	Defects found (typical)
Informal code review	25%
Unit tests	30%
Integration tests	35%
Formal code inspection	60%

- No single technique finds most defects: combine them
- Microsoft (Bacchelli & Bird, 2013) and Google (Sadowski et al., 2018): review finds bugs, but brings even more knowledge transfer and better code

> Review and tests are complementary: today we do both.

The bus factor

How many people must be hit by a bus
before the project stops?

The bus factor

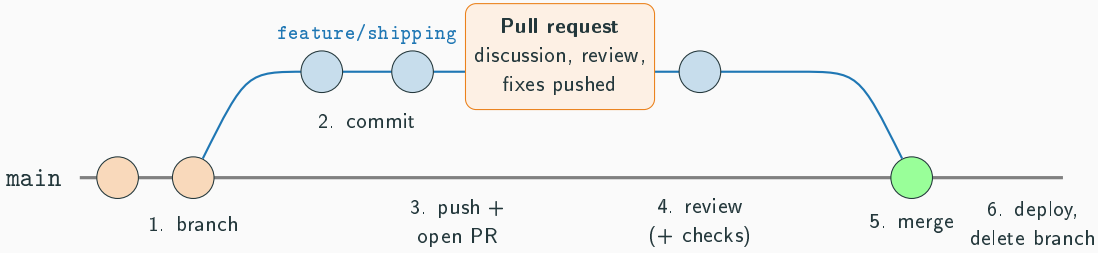
How many people must be hit by a bus
before the project stops?

- Bus factor = 1: only one person understands the payment code
- They are sick, on holiday, or leave the company → nobody can fix a bug in production
- Every review spreads knowledge: the reviewer now knows this code

> Code review is the cheapest way to raise the bus factor: knowledge spreads as a side effect of daily work.

GitHub flow and pull requests

From branch to pull request



- A pull request (PR): "please pull my branch into `main`", plus a place to discuss it
- Commits pushed to the branch update the PR automatically

> A pull request turns a branch into a team decision: nobody merges alone.

GitHub flow (S. Chacon, 2011)

1. `main` is always deployable
2. Create a descriptively named branch from `main` (`fix/discount-limit`)
3. Commit locally, push the branch often
4. Open a pull request when you want feedback, or when you are done
5. Merge after review and green checks
6. Deploy right after the merge; delete the branch

> Short-lived branches + a PR as the gate to `main`: the trunk-based idea of lecture 2, made safe.

Anatomy of a pull request

- Title: what, in one line (like a commit subject)
- Description: why, how to test, screenshots
- Linked issue: `Closes #12`
- Reviewers: who must approve
- Files changed: the diff, with line comments
- Conversation: questions, answers, decisions
- Checks: automated tests (next session)
- Merge button: only when everything is green

> In 6 months, the PR is where the "why" survives, next to the commit messages.

Issues: the “why” before the code

```
Issue #3   Discount over 100% gives negative prices
  apply_discount([100], 150) returns [-50.0].
  Expected: the discount is refused.

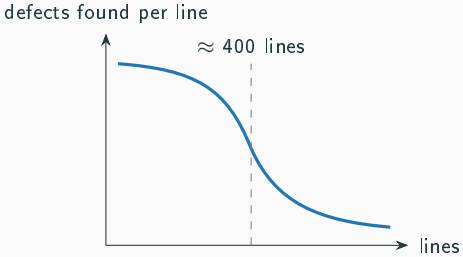
Pull request #4   Refuse discounts outside 0-100%
  Closes #3
```

- An issue describes a problem or a request: bug, feature, question
- `Closes #3` in a PR description: the issue closes automatically when the PR is merged

> Issue → branch → PR → merge: every change in `main` can be traced back to a reason.

Small pull requests

- Cisco study (SmartBear, 2006, 2500 reviews): beyond about 400 lines, reviewers find much fewer defects
- 10 lines: 10 comments. 500 lines: “Looks good to me”
- One PR = one purpose: no “fix + refactoring + new feature”



> Lecture 1 again: small batches are easier to check, faster to merge, and cheaper to undo.

Three merge buttons

Button	Result on `main`	Good for
Merge commit	All commits of the branch + a merge commit	Keeping the full real history
Squash and merge	One commit with all the changes of the PR	Clean history: one PR = one commit
Rebase and merge	The commits replayed on top of `main`, no merge commit	Linear history, commits kept

- Same choice as lecture 2 (merge or rebase?), now made on GitHub
- Squash: the "wip" and "fix typo" commits of the branch disappear from `main`

> Pick one convention per project and write it down. In our labs: squash and merge.

Protecting `main`

A branch rule (GitHub ruleset) on `main`:

- Require a pull request: no direct push
- Require 1 approval: the author cannot approve their own PR
- Block force pushes and deletion
- Require status checks: tests must pass (next session)

```
$ git push origin main
remote: error: GH013: Repository rule
  violations found for refs/heads/main.
remote: - Changes must be made
  through a pull request.
 ! [remote rejected] main -> main
```

> A process that can be skipped will be skipped, the evening before a release. Make the safe path the only path, for everyone, admins included.

Code review in practice

What to look for, what to leave to machines

Humans

- Does it do what the issue asks?
- Missing cases: empty, zero, negative, too big
- Are there tests for the new behaviour?
- Can I understand it without the author?
- Is it the right design?

Machines

- Formatting, spaces, line length (formatter)
- Unused variables, style rules (linter)
- "Do the tests still pass?" (CI)
- Known vulnerable libraries (scanner)

> Do not spend human attention on what a tool can check: reviewers should argue about behaviour and design, not commas.

Writing review comments

Bad

- "This is wrong."
- "Why would anyone write this?"
- "Use a list comprehension." (on a style choice)
- 40 comments on spaces

Good

- "What happens with `percent = 150`? Could we add a test?"
- "I find `shipping_cost` clearer than `sc`, what do you think?"
- "nit: typo in the docstring" (minor, optional)
- "Nice idea, I did not know this function!"

- Comment the code, never the person; ask questions; explain why
- Say clearly what blocks the merge and what is only a suggestion

Being reviewed

- Read your own diff on GitHub before asking for review: you will find half of the problems
- Keep the PR small, with a description that explains why and how to test
- Answer every comment: fix it, or explain why not
- A comment on your code is not an attack on you: you are not your code
- Disagreement? Talk (or call) instead of a 30-message thread

> Good review is a conversation between two people who want the same thing: working code in `main`.

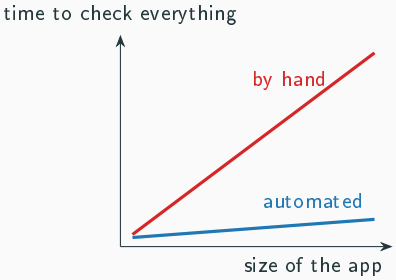
Why tests?

“It works, I tried it”

- You tried it once, with 10%, on your laptop
- Next week, someone changes `total()`: who tries `apply_discount` again?
- 50 functions $\times$ 10 changes a week: nobody re-checks everything by hand

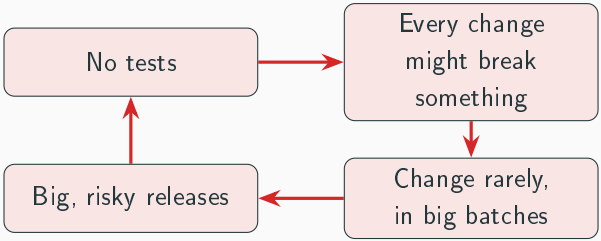
“It works, I tried it”

- You tried it once, with 10%, on your laptop
- Next week, someone changes `total()`: who tries `apply_discount` again?
- 50 functions × 10 changes a week: nobody re-checks everything by hand



> An automated test is a check written once and run thousands of times, in seconds, by anyone, and soon by a machine (CI).

Fear of change

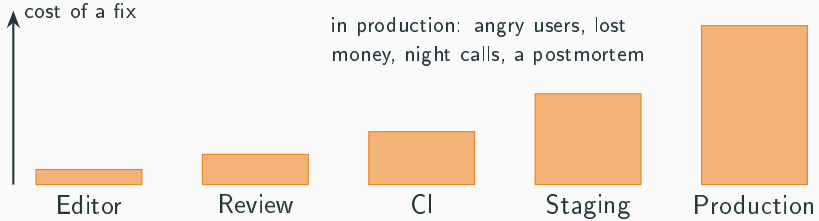


With tests
change → run the tests
→ green: ship it
→ red: you know *where*

- Lecture 1's vicious circle, at the scale of one function
- Without tests, nobody dares to clean up (refactor) old code: it only gets worse

> Tests are not about proving the code is perfect: they give the team the courage to change it.

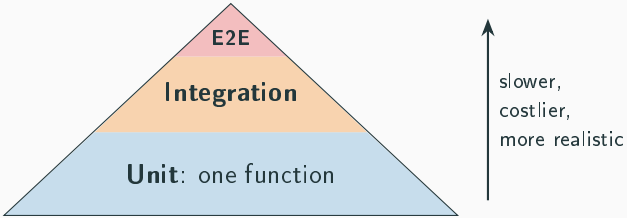
The later a bug is found, the more it costs



- Shift left: move the checks as early as possible (tests, review, then CI)

> Each practice of this course moves the discovery of a bug one step to the left.

The test pyramid (M. Cohn, 2009)



- Unit: one function, milliseconds, hundreds of them
- Integration: several parts together (app + database)
- End-to-end: a robot clicks through the real app, slow and fragile

Many fast unit tests at the bottom, a few slow end-to-end tests at the top. Today: unit tests.

Writing tests with pytest

A first test

```
# test_cart.py
from cart import average

def test_average():
    # arrange
    prices = [10, 20]
    # act
    result = average(prices)
    # assert
    assert result == 15
```

- A test is a function whose name starts with `test_`, in a file `test_*.py`
- Arrange, act, assert
- pytest finds and runs them all

```
$ pytest
test_cart.py ....          [100%]
====== 4 passed in 0.02s ======
```

Reading a failure

```
$ pytest
test_cart.py .F..                                                    [100%]
_________________________ test_average_empty_cart _________________________

    def test_average_empty_cart():
>       assert average([]) == 0
E       assert None == 0
E        +  where None = average([])

test_cart.py:7: AssertionError
==================== 1 failed, 3 passed in 0.03s ====================
```

- Which test failed, on which line, with which values: `average([])` returned `None`, not 0
- A red test is good news: the bug was found on your laptop, not by a customer

What to test?

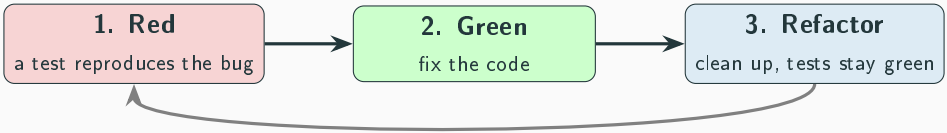
Kind of case	Example for the shop
Normal case	`average([10, 20]) == 15`
Empty / zero	`average([]) == 0`, `total([]) == 0`
Boundaries	Free shipping "from 50": test 49, 50, 51
Invalid input	`apply_discount([100], 150)` must raise an error

```
import pytest

def test_discount_over_100_is_refused():
    with pytest.raises(ValueError):
        apply_discount([100], 150)
```

> Bugs live at the edges: empty, zero, boundaries, invalid input. Test those first.

A bug? Write the test first



- The issue says "150% gives negative prices": first, a test that fails for this reason
- Then fix: the test turns green. You know the fix works
- The test stays forever: this bug can never come back silently (regression test)
- Writing tests before the code for everything: Test-Driven Development (K. Beck, 2002)

> Never fix a bug without a test that proves it: otherwise it will come back.

Coverage: a measure, not a goal

- Coverage: % of lines run by the tests (`pytest --cov`, plugin `pytest-cov`)
- Low coverage: whole parts are never tested. Useful alarm
- High coverage does not mean good tests

```
def test_discount():
    apply_discount([100], 10)
    # 100% coverage of
    # apply_discount...
    # and no assert!
```

> Goodhart's law: "when a measure becomes a target, it ceases to be a good measure". Aim for tests that would fail if the code broke, not for a number.

Good unit tests: FIRST

Property	Why
Fast	Milliseconds: you run them after every change, without thinking
Independent	Any order, alone or together: one test never depends on another
Repeatable	Same result on every laptop, every day (no network, no clock)
Self-validating	Pass or fail, no human reading the output
Timely	Written with the code (or before), not "later"

A slow or random ("flaky") test is soon ignored by everyone: it is worse than no test.

Wrap-up

Back to the broken production

Practice	What it would have done
Issue	Written down the rule: "a discount is between 0 and 100%"
Protected `main`	No direct push the evening before: a PR is mandatory
Code review	"What happens with 150?"
Test with `pytest.raises`	Red on the author's laptop, before the PR

> Two cheap safety nets: a human who reads, a machine that checks. Each one catches what the other misses.

Key takeaways

1. Review catches bugs, but above all spreads knowledge (bus factor) and builds shared ownership
2. GitHub flow: short branch → pull request → review → merge; `main` is always deployable
3. Small PRs, linked to an issue, reviewed with kind and precise comments
4. Protect `main`: the safe path must be the only path
5. Automated tests replace "I tried it once"; they give the courage to change
6. Test the edges; for each bug, write the failing test first

Today's lab and further reading

Lab 2

- First pytest tests for the shop
- Protect `main`
- Issue → PR → review → merge, each student in both roles
- Fix the discount bug, test first

Reading

- GitHub Docs, "GitHub flow", docs.github.com
- Google, *Engineering Practices: Code Review Developer Guide*, google.github.io/eng-practices
- *pytest* documentation, "Get Started", docs.pytest.org
- M. Fowler, "TestPyramid", martinfowler.com, 2012

Next session

Continuous Integration

Who runs the tests? Every time? On every branch?
How can a machine check each pull request before a human reads it?